CHAPTER 2 · APPLICATION LAYER

2.7 Socket Programming

Creating Network Applications

Kurose & Ross, 8th Ed.

INTRO

Today's Lecture — Roadmap

Chapter 2.7 — Socket Programming

1. **What is a Socket?**
   The API between your application and the network. The door analogy.

2. **Two Types of Network Apps**
   Open (RFC-defined, well-known ports) vs. Proprietary (custom protocol).

3. **UDP or TCP? — First Decision**
   Key differences: reliability, connection, overhead.

4. **UDP Socket Programming**
   UDPClient.py and UDPServer.py — complete line-by-line walkthrough.

5. **TCP Socket Programming**
   TCPClient.py and TCPServer.py — welcoming socket, connection socket, handshake.

SOCKETS

What Is a Socket?

Section 2.7 — Introduction

A socket is the interface between the application layer and the transport layer — the API for building network applications.

The Door Analogy

A process is a house.
A socket is the house's door.

To send data: push it out through your socket door.
To receive data: data arrives at your socket door.

The transport layer (TCP/UDP) is the delivery infrastructure outside the door.

Developer Control

Application side of the socket:
-> Developer controls EVERYTHING

Transport side of the socket:
-> Developer controls almost NOTHING

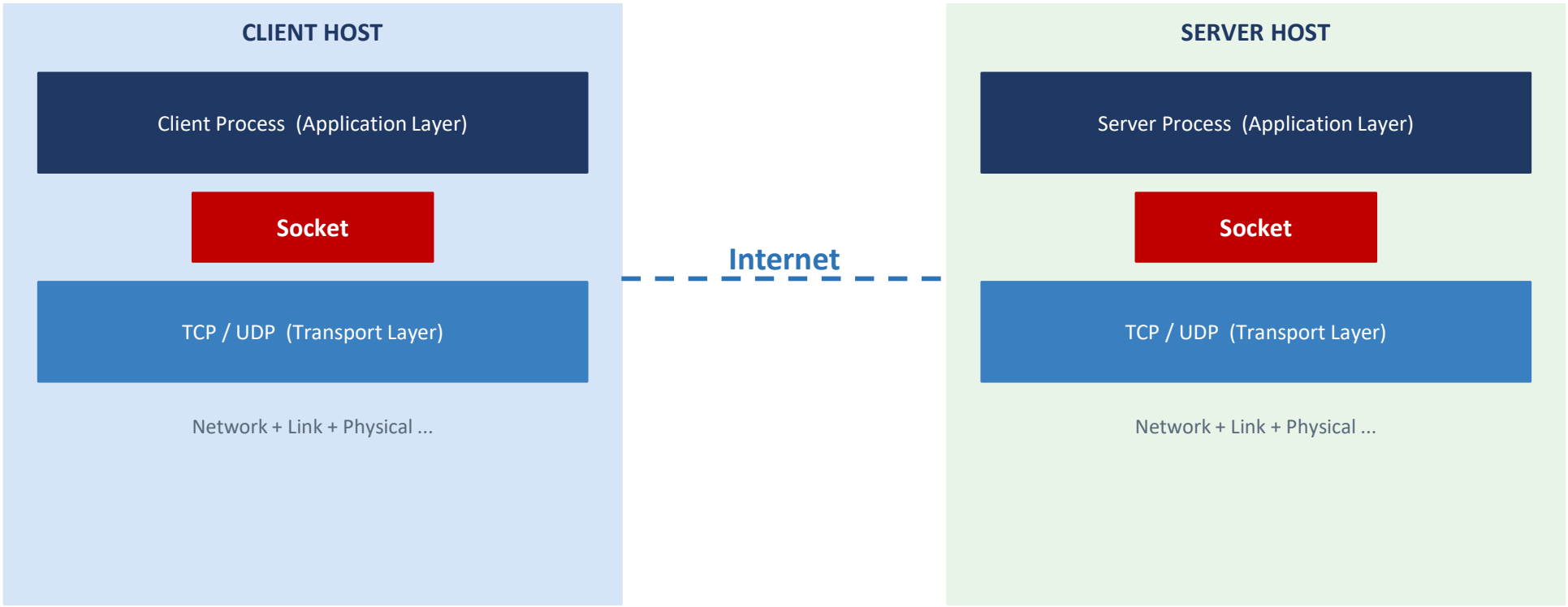
Only two choices on the transport side:
(1) TCP or UDP
(2) A few parameters: buffer size, segment size

The socket abstraction hides enormous network complexity from the application developer.

SOCKETS

The Client-Server Application Structure

Section 2.7 — Introduction



DESIGN

Two Types of Network Applications

Section 2.7 — Introduction

Open Applications

Protocol specified in a public RFC (IETF standard).

Client & server by independent teams — both follow RFC -> interoperability.

Examples:
Chrome <-> Apache (HTTP, RFC 2616)
Outlook <-> Gmail (SMTP, RFC 5321)

Must use well-known port numbers:
HTTP: 80 SMTP: 25 DNS: 53

Proprietary Applications

Protocol NOT publicly documented.

Single team builds both client and server — full protocol control.

No outside developer can build a compatible client.

Must AVOID well-known port numbers.
Use ports 1024-65535.

Our example today is proprietary:
capitalize service on port 12000.

Open = interoperability via RFC standard | Proprietary = full control, no outside interop

DESIGN

First Design Decision: UDP or TCP?

Section 2.7 — Transport Protocol Choice

Choosing the transport protocol is the first decision any network application developer must make.

TCP (SOCK_STREAM)

Connection-oriented: 3-way handshake before data flows.

Reliable byte-stream: every byte arrives, in order.

Flow control + congestion control.

Higher overhead and latency.

Best for: HTTP, SMTP, FTP, SSH

UDP (SOCK_DGRAM)

Connectionless: no setup, no teardown.

No reliability guarantee: packets can be lost, reordered, or duplicated.

No flow control, no congestion control.

Low overhead, low latency.

Best for: DNS, streaming, online gaming

We implement BOTH versions today using the same application logic — compare the structural differences.

DEMO APP

The Demo Application: Capitalize Service

Section 2.7 — Our Working Example

Client sends a sentence -> Server capitalizes it -> Server sends it back -> Client displays it.

1. **Client reads input**
   User types a lowercase sentence at the keyboard.

2. **Client sends to server**
   The sentence is sent through the socket to the server.

3. **Server capitalizes**
   Server receives and converts all characters to UPPERCASE.

4. **Server replies**
   The capitalized sentence is sent back through the server's socket.

5. **Client displays result**
   Client receives and prints the capitalized sentence.

Simple logic — but contains ALL the essential mechanics of any client-server application.

SECTION 2.7.2

Socket Programming with UDP

Connectionless · No reliability guarantee · Low overhead, low latency

UDP

UDP: Addressing and the Packet Model

Section 2.7.1 — Socket Programming with UDP

With UDP, every packet carries its own destination address: (IP address, port number).

Destination address = IP + Port

IP address -> routes the packet to the correct HOST (via routers).

Port number -> tells the OS which SOCKET (which process) receives the packet.

Together: (IP, port) uniquely identifies a socket anywhere on the Internet.

Source address — automatic

The sending process also has an IP + port (source address).

The OS attaches it to outgoing packets AUTOMATICALLY — the app does NOT do this.

The server reads the source address from incoming packets to know WHERE TO SEND THE REPLY.

Server port: assigned EXPLICITLY by the developer using bind() — must be fixed and known to clients.
Client port: assigned AUTOMATICALLY by the OS from the ephemeral range (49152-65535).

UDPClient.py — Complete Code

Section 2.7.1 — Socket Programming with UDP

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_DGRAM)
message = input('Input lowercase sentence:')
clientSocket.sendto(message.encode(), (serverName, serverPort))
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

1 *Import socket module*
2 *Server name or IP*
3 *Server port (12000)*
4 *Create UDP socket*
5 *Read user input*
6 *Send datagram*
7 *Block and receive reply*
8 *Print result*
9 *Close socket*

9 lines of Python = a complete UDP client. The OS assigns the client port automatically.

UDP

UDPClient.py — Key Lines Explained

Section 2.7.1 — Socket Programming with UDP

socket(AF_INET, SOCK_DGRAM)

Name = AF_INET
Value = IPv4 address family

```
SOCK_DGRAM -> UDP socket
No port specified -> OS assigns one
```

sendto(msg, (host, port))

Name = msg.encode()
Value = convert string -> bytes

```
(serverName, serverPort) =
destination
Source addr auto-attached by OS
```

recvfrom(2048)

Name = modifiedMessage
Value = received bytes

```
serverAddress = reply origin
2048 = receive buffer size
```

Client Execution Flow

UDP

UDPServer.py — Complete Code

Section 2.7.1 — Socket Programming with UDP

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print('The server is ready to receive')
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(), clientAddress)
```

1 *Import socket module*
2 *Fixed server port*
3 *Create UDP socket*
4 *bind() to port 12000 <- KEY*
5 *Ready message to console*
6 *Infinite loop*
7 *Block for incoming packet*
8 *Capitalize: application logic*
9 *Reply to client address*

bind() is the key difference: server explicitly registers port 12000 so clients can find it.

UDP

UDPServer.py — Key Lines Explained

Section 2.7.1 — Socket Programming with UDP

serverSocket.bind(('', port))

Name = '' (empty string)
Value = all network interfaces on host

```
port 12000 -> any incoming UDP
packet to port 12000 -> this socket
```

recvfrom(2048) returns clientAddress

Name = message
Value = packet data (bytes)

```
clientAddress = (client_IP, port)
Used as the reply destination
```

Key Properties of the UDP Server

- Runs in an INFINITE LOOP — serves requests indefinitely, no shutdown.
- STATELESS: each received packet is independent. After replying, the server forgets the client entirely.
- Learns the client return address from the packet source field — used directly in sendto().
- No connection setup or teardown — each UDP exchange is a self-contained transaction.

SECTION 2.7.2

Socket Programming with TCP

Connection-oriented · Reliable byte-stream · Two socket types · Invisible handshake

TCP

TCP: The Connection Model

Section 2.7.2 — Socket Programming with TCP

TCP requires a CONNECTION to be established BEFORE any data flows.

1. **Server creates welcoming socket**
   Server opens a socket and calls listen() — waits for connection requests.
2. **Client initiates connection**
   Client calls connect(serverIP, serverPort) — triggers the 3-way handshake.
3. **3-way handshake (invisible)**
   SYN -> SYN-ACK -> ACK happens inside TCP in the OS. App code never sees it.
4. **Server accepts — new socket**
   Server's accept() returns a NEW dedicated connectionSocket for this client.
5. **Data exchange**
   Both sides exchange data via send() and recv() through their sockets.
6. **Connection close**
   Client calls close() -> TCP sends FIN message -> connection terminated gracefully.

TCP

TCP Server: Two Socket Types

Section 2.7.2 — Socket Programming with TCP

The TCP server needs TWO types of sockets — a critical difference from UDP.

serverSocket — The Welcoming Socket

Created once at startup.

Listens for incoming connection requests from any client.

Does NOT exchange application data.

Analogy: the hotel reception desk.
Every guest arrives here first.
Front desk stays open always.

connectionSocket — The Connection Socket

Created by accept() for EACH new client.

Dedicated exclusively to one client conversation.

All application data flows here.

Closed when the conversation ends.
serverSocket stays open for the next client.

Common mistake: confusing the welcoming socket with the connection socket. They are distinct objects.

TCP

TCPClient.py — Complete Code

Section 2.7.2 — Socket Programming with TCP

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print('From Server: ', modifiedSentence.decode())
clientSocket.close()
```

1 Import socket module

2 Server name or IP

3 Server port

4 SOCK_STREAM = TCP <- KEY

5 connect() -> 3-way handshake

6 Read user input

7 send() — no addr needed!

8 Receive reply

9 Print result

10 close() -> sends TCP FIN

SOCK_STREAM = TCP socket. connect() triggers the invisible handshake. send() needs no address.

TCP

TCPClient.py — Key Differences from UDP

Section 2.7.2 — Socket Programming with TCP

SOCK_STREAM vs SOCK_DGRAM

Name = SOCK_STREAM
Value = TCP — reliable byte stream

Only this one parameter changes to switch from UDP to TCP socket

connect((host, port))

Name = serverName, serverPort
Value = welcoming socket address

Triggers 3-way handshake App never sees SYN/ACK

send() vs sendto()

Name = send(data)
Value = no destination tuple needed

Connection already knows dest. No (IP, port) required per msg

Why send() instead of sendto()?

With TCP, the connection carries the destination information permanently. Once connect() succeeds, the OS knows the destination for all subsequent data. There is no need to specify (IP, port) per message — unlike UDP where every datagram is independent and must carry its own destination.

TCP

TCPServer.py — Complete Code

Section 2.7.2 — Socket Programming with TCP

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()
```

1 Import socket module

2 Fixed server port

3 Create TCP socket

4 Bind to port 12000

5 listen() — accept conns

6 Ready message

7 Infinite loop

8 accept() -> new socket <- KEY

9 Receive data from client

10 Capitalize

11 Send reply

12 Close connectionSocket only

TCP

TCPServer.py — Key Lines Explained

Section 2.7.2 — Socket Programming with TCP

serverSocket.listen(1)

Name = parameter
Value = max queued connections

Marks socket as passive.
Ready to accept incoming connections.

connectionSocket, addr = serverSocket.accept()

Name = connectionSocket
Value = new socket dedicated to client

Blocks until a client connects.
Creates a NEW socket per client.

TCPServer Lifecycle

serverSocket stays open after each connectionSocket.close() — ready for the next client.

TCP

The Three-Way Handshake — Invisible to the App

Section 2.7.2 — TCP Connection Mechanics

The 3-way handshake happens entirely inside the TCP layer in the OS — the application never sees it.

Layering in action: app uses connect() / accept() — the TCP layer handles the handshake mechanics.

COMPARISON

UDP vs. TCP — Side-by-Side Comparison

Section 2.7 — Summary of Differences

UDP thinks in packets. TCP thinks in streams.

Aspect	UDP (SOCK_DGRAM)	TCP (SOCK_STREAM)
Connection setup	None	3-way handshake
Reliability	None – best effort	Guaranteed, in-order
Socket type	SOCK_DGRAM	SOCK_STREAM
Send method	sendto(msg, (host, port))	send(msg)
Receive method	recvfrom(buf)	recv(buf)
Server sockets	One socket for all clients	Two: serverSocket + connectionSocket
Server bind?	Required	Required
Client bind?	OS assigns automatically	OS assigns automatically
Use cases	DNS, streaming, gaming	HTTP, SMTP, FTP, SSH

PRACTICE

Port Numbers — Practical Reference

Section 2.7 — Practical Notes

Well-Known Ports (0-1023)

Reserved for standard services.
Must match the specified protocol.

HTTP -> 80
HTTPS -> 443
SMTP -> 25
DNS -> 53
FTP -> 21
SSH -> 22

Registered Ports (1024-49151)

Less strictly controlled.
Applications may register with IANA.

MySQL -> 3306
HTTP alt -> 8080
Redis -> 6379

Avoid unless your app specifically needs a registered port.

Ephemeral Ports (49152-65535)

Dynamic / private range.
Free for any use.

OS assigns client ports automatically from this range.

Our demo uses port 12000 (registered range, fine for a proprietary app).

Proprietary app: pick any port 1024-65535 not already in use. Avoid 0-1023 (reserved).

REVIEW

Review Questions

Section 2.7 — Kurose & Ross

Sockets & Design

What is a socket? What is the relationship between a socket and a port number?

What are the two types of network apps? How do they differ in port usage?

What is the difference between SOCK_DGRAM and SOCK_STREAM?

Why is it safe to let the OS assign the client port but not the server port?

UDP & TCP Specifics

Why does a UDP server need bind() but a UDP client does not?

How does the UDP server know where to send its reply?

What is the difference between serverSocket and connectionSocket in TCPServer?

What triggers the 3-way handshake? Is it visible to the application?

What is the difference between send() and sendto()?

What happens to serverSocket after connectionSocket.close()?

Ref: Kurose & Ross, 8th Ed. — Chapter 2.7, pp. 152-165 | Socket Programming Assignments pp. 175-177

Summary

2.7 — Socket Programming: Creating Network Applications

- A socket is the API between the application layer and the transport layer — the developer's door to the network.
- Developer controls the app side of the socket; OS controls the transport side. Two choices: TCP or UDP.
- Open apps follow RFCs and use well-known ports. Proprietary apps define their own protocol and avoid reserved ports.
- UDP (SOCK_DGRAM): connectionless, no reliability. Each packet carries its own destination via sendto().
- UDP server: bind() registers the port; recvfrom() returns the client address used to reply via sendto().
- TCP (SOCK_STREAM): connection-oriented; connect() on the client triggers the invisible 3-way handshake.
- TCP server has TWO sockets: serverSocket (welcoming, waits) and connectionSocket (per-client, created by accept()).
- TCP uses send() with no destination per message — the connection itself carries the routing information.
- The 3-way handshake is completely transparent to the application — it happens inside the OS TCP layer.
- Ports 0-1023 are reserved. Client ports auto-assigned by OS. Server ports must be fixed and bound explicitly.
- A complete functional client-server app in Python: fewer than 10 lines of code per side.